

Manuale Tecnico

Indice

[Struttura](#)

[Progettazione](#)

[Scelte Progettuali](#)

[Scelte Architettureali](#)

[Protocollo Client-Server](#)

[Gestione della Concorrenza](#)

[Strutture Dati](#)

[Pattern](#)

[Limiti della soluzione sviluppata](#)

[Sitografia](#)

Struttura

TheKnife è un monorepo Maven multi-modulo composto da tre moduli: theknife-common (DTO e protocollo condivisi), theknife-backend (server Quarkus con persistenza PostgreSQL) e theknife-frontend (client desktop JavaFX). Client e server comunicano tramite un unico endpoint WebSocket scambiando messaggi JSON.

Requisiti Tecnici

- Java Development Kit (JDK) 21 — necessario per compilare ed eseguire i tre moduli;
- Apache Maven 3.8 o superiore (oppure il Maven Wrapper incluso) per la gestione delle dipendenze e la build;
- Docker e Docker Compose per l'esecuzione del database PostgreSQL (e, opzionalmente, del tunnel ngrok per la demo remota);
- PostgreSQL 13 o superiore — fornito tramite container Docker;
- JavaFX 23.0.1 — gestito automaticamente da Maven per l'esecuzione del client.

Processore dual-core, minimo 512 MB di memoria per il backend e spazio su disco sufficiente per il database e per il dataset di seed (circa 18 MB di dati Michelin, ~17.700 ristoranti).

Esecuzione del progetto

1. `avvia_postgres`
 - a. avvia PostgreSQL (porta 5432).
2. `avvia_ngrok`
 - a. avvia ngrok per il tunneling. NOTA: impostare variabile d'ambiente `NGROK_AUTHTOKEN` per il token di autenticazione.
3. `avvia_server`
 - a. avvia il backend (porta 8080). Al primo avvio, se il database è vuoto, i dati vengono importati da `"data/"`.
4. `avvia_client`
 - a. avvia il client. Come indirizzo del server usare `ws://localhost:8080`. L'indirizzo può essere passato come argomento (es. `./avvia_client.sh ws://localhost:8080`) oppure inserito nella finestra che appare all'avvio.

Uso su Windows: eseguire i file `.bat`

Uso su Linux/macOS: eseguire i file `.sh` (es. `./avvia_server.sh`)

Note:

- `avvia_client.sh` sceglie automaticamente il jar giusto per il sistema operativo/architettura. Per architetture non incluse (es. Linux arm64) ricompilare il frontend: `mvn -pl theknife-frontend -am package -Djavafx.platform=linux-aarch64`
- I jar del client contengono le librerie native JavaFX della sola piattaforma indicata nel nome.

Progettazione

Scelte Progettuali

Database

Tabella Users

- `username VARCHAR(255) PRIMARY KEY`
- `first_name VARCHAR(255), last_name VARCHAR(255)`
- `password VARCHAR(255) NOT NULL — hash BCrypt`
- `birth_date DATE, city VARCHAR(255)`
- `role VARCHAR(20) NOT NULL — 'CLIENTE' | 'RISTORATORE'`

Tabella Restaurants

- `id VARCHAR(64) PRIMARY KEY — hash legacy per i dati di seed, UUID per i nuovi`
- `name, address, location, price, cuisine, phone, longitude, latitude`
- `michelin_url, website_url, award, green_star, facilities, description`

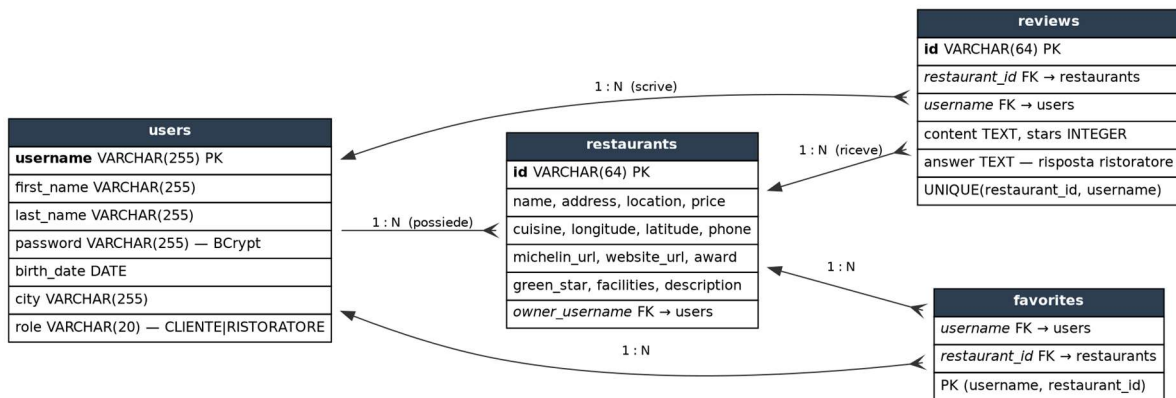
- owner_username VARCHAR(255) REFERENCES users(username) ON DELETE SET NULL

Tabella Reviews

- id VARCHAR(64) PRIMARY KEY
- restaurant_id REFERENCES restaurants(id) ON DELETE CASCADE
- username REFERENCES users(username) ON DELETE CASCADE
- content TEXT, stars INTEGER, answer TEXT
- UNIQUE(restaurant_id, username) — una recensione per utente per ristorante

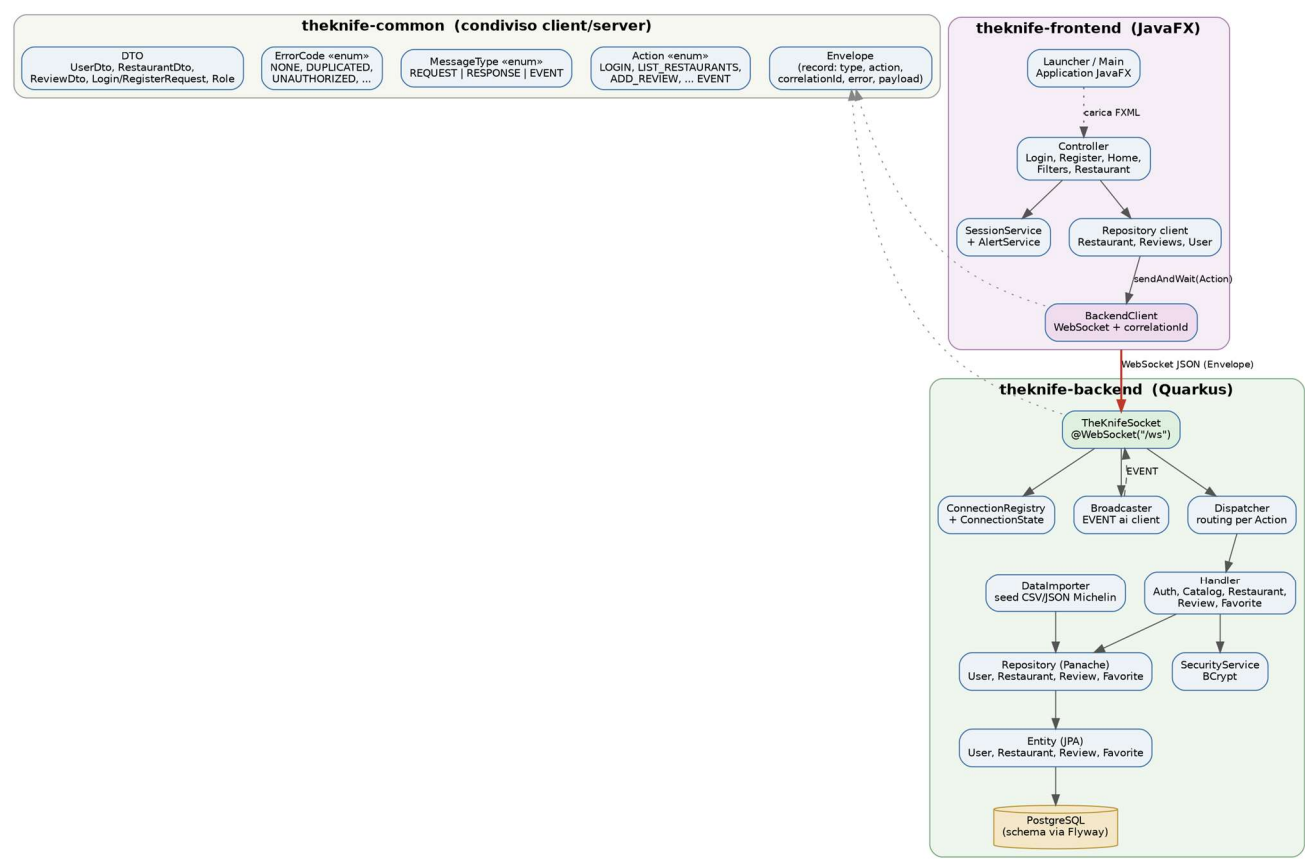
Tabella Favorites

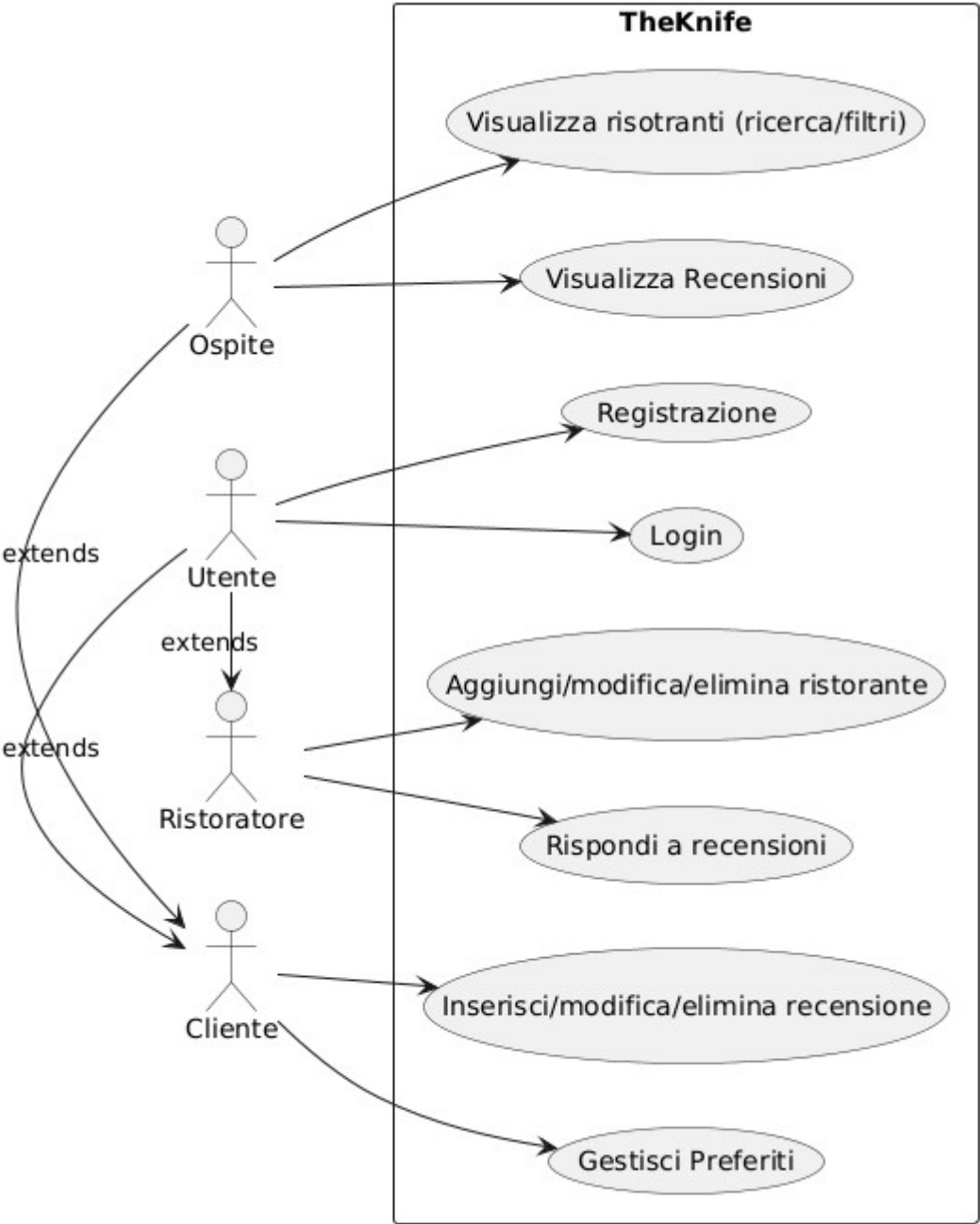
- username REFERENCES users(username) ON DELETE CASCADE
- restaurant_id REFERENCES restaurants(id) ON DELETE CASCADE
- PRIMARY KEY (username, restaurant_id) — tabella associativa a chiave composta



Diagrammi UML

Class Diagram

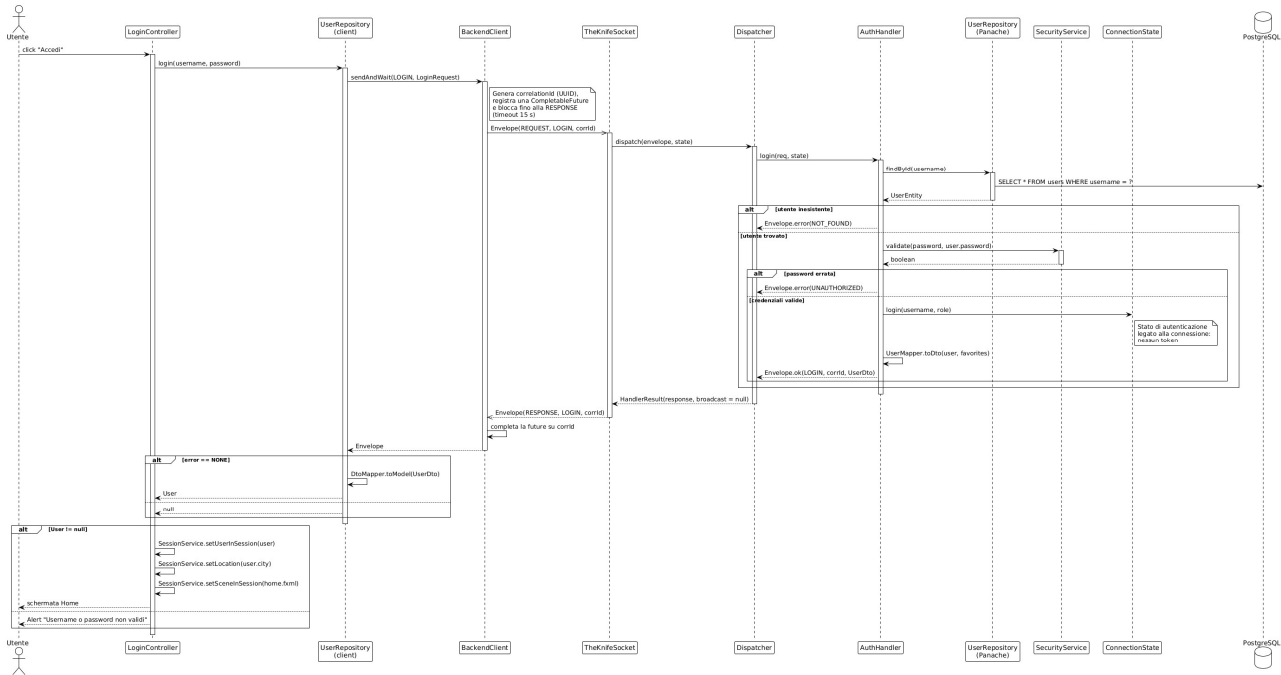




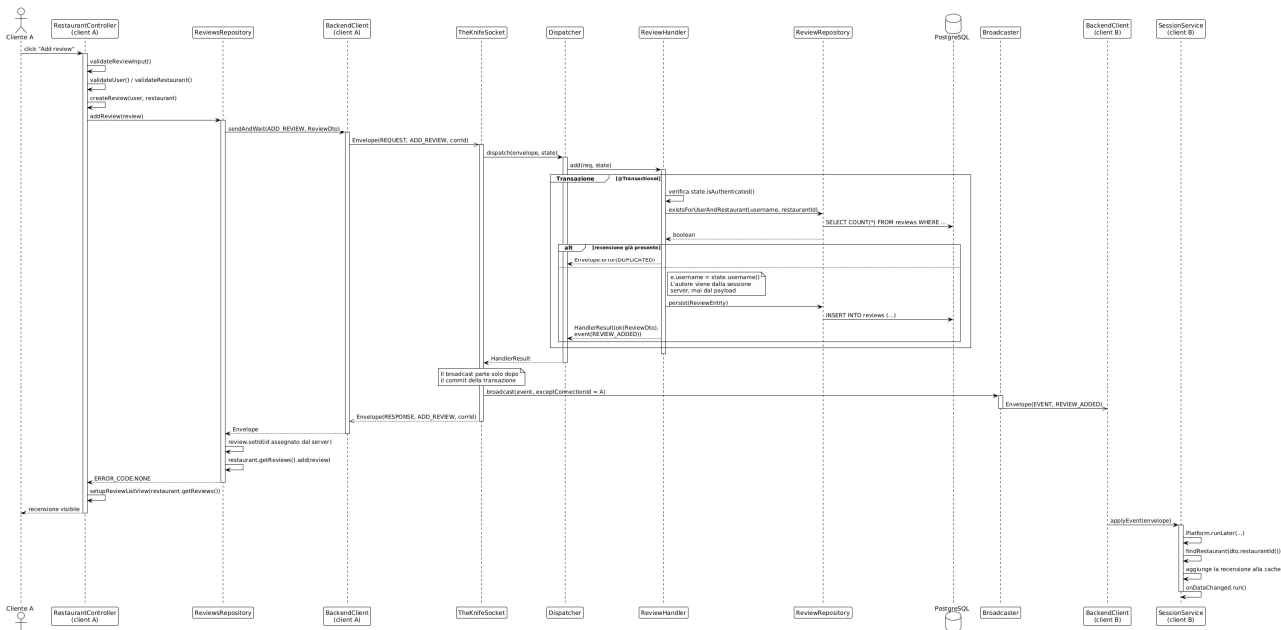
TheKnife

Alessio Luciano, Marcello Mordente, Luca Morosini, Luca Nardo, 20/7/2026, Versione 2.0

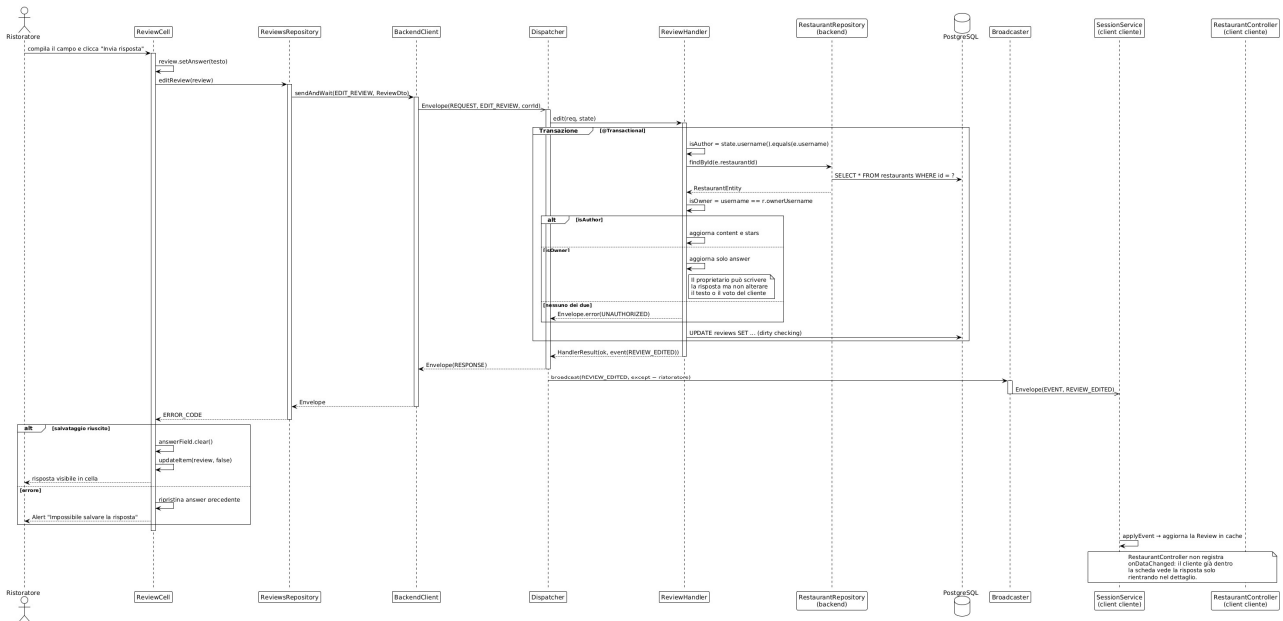
Login Sequence Diagram



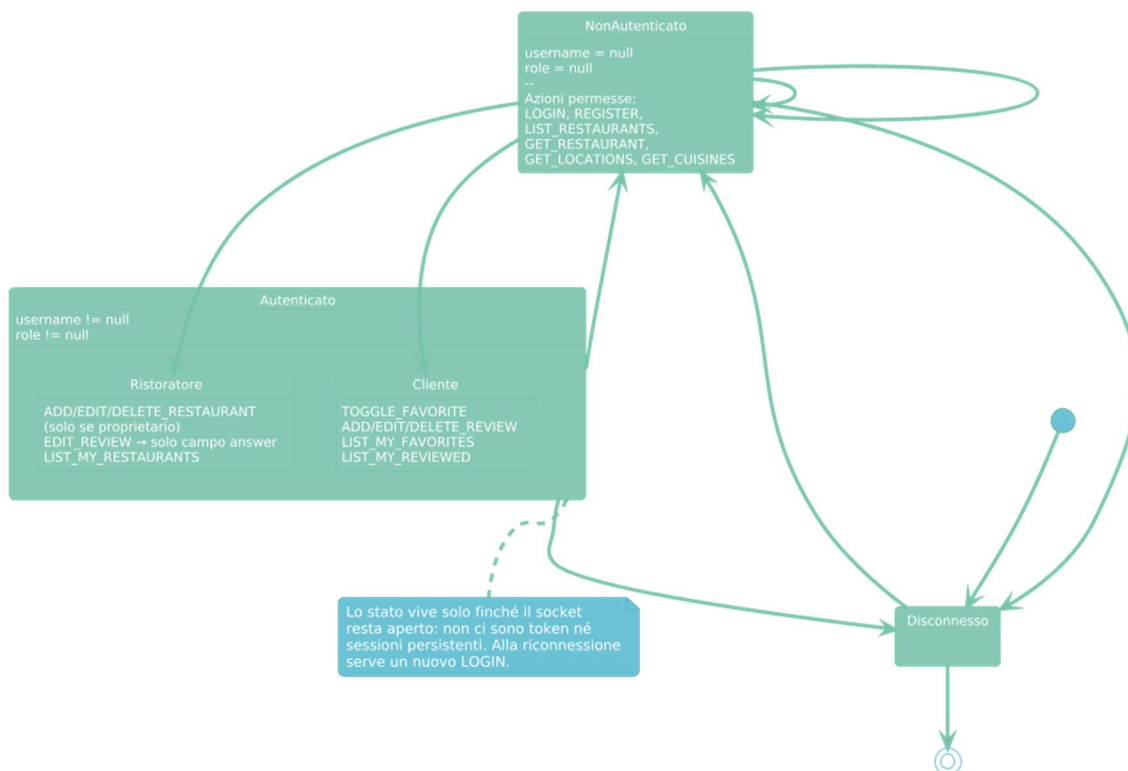
Sequence Diagram Recensioni



Sequence Diagram Risposte alle Recensioni



State Diagram Connessione



Scelte Architetture

TheKnife adotta un'architettura client/server in cui i moduli comunicano tramite un unico endpoint WebSocket. La separazione in tre moduli (common, backend, frontend) garantisce una netta distinzione delle responsabilità e la condivisione del protocollo tra le due parti.

Architettura lato server

Il backend, basato su Quarkus, è il nodo centrale del sistema ed è responsabile di:

- accettare le connessioni WebSocket in ingresso sull'endpoint /ws;
- interpretare i messaggi (Envelope) e instradarli all'handler competente tramite il Dispatcher;
- coordinare l'accesso al database tramite repository Panache in transazione;
- notificare in tempo reale gli altri client connessi (broadcast di eventi) dopo ogni modifica andata a buon fine;
- mantenere lo stato di autenticazione per singola connessione.

Architettura lato client

Il client JavaFX si occupa di:

- stabilire e mantenere la connessione WebSocket con il backend (classe BackendClient);
- inviare richieste e ricevere risposte correlate tramite correlationId;
- gestire l'interazione con l'utente tramite interfaccia grafica FXML (un controller per schermata);
- reagire agli eventi in tempo reale ricevuti dal server aggiornando le viste.

Protocollo Client-Server

La comunicazione avviene su socket WebSocket (TCP) con un protocollo applicativo JSON. Il modello è connection-oriented: un'unica connessione persistente per client trasporta richieste, risposte ed eventi. Il server accetta più connessioni simultanee.

Envelope

Ogni messaggio è un Envelope JSON con i campi: type (REQUEST, RESPONSE o EVENT), action (l'operazione o l'evento), correlationId (UUID impostato sulla REQUEST ed eco sulla RESPONSE; assente sugli EVENT), error (codice di esito sulla RESPONSE), message (dettaglio testuale opzionale) e payload (corpo JSON specifico dell'azione).

I tipi di messaggio (MessageType): REQUEST client → server, RESPONSE server → client (eco del correlationId), EVENT broadcast dal server (senza correlationId).

Action

L'enum Action definisce tutte le operazioni e gli eventi:

- Autenticazione: LOGIN, REGISTER, LOGOUT;
- Consultazione: LIST_RESTAURANTS, LIST_MY_RESTAURANTS, GET_RESTAURANT, GET_LOCATIONS, GET_CUISINES;
- Ristoranti (ruolo RISTORATORE): ADD_RESTAURANT, EDIT_RESTAURANT, DELETE_RESTAURANT;
- Recensioni: ADD_REVIEW, EDIT_REVIEW, DELETE_REVIEW;

- Preferiti (ruolo CLIENTE): TOGGLE_FAVORITE;
- Eventi broadcast (server → client): RESTAURANT_ADDED/EDITED/DELETED, REVIEW_ADDED/EDITED/DELETED.

ErrorCode

Gli errori sono restituiti nella RESPONSE tramite l'enum ErrorCode: NONE, DUPLICATED, NOT_FOUND, UNAUTHORIZED, VALIDATION, SERVICE_ERROR, accompagnati da un messaggio leggibile. Il client mappa tali codici sulle proprie enum interne, mantenendo la connessione attiva quando possibile.

Gestione della Concorrenza

Modello di concorrenza

Il backend è progettato per servire più client simultaneamente. Ogni connessione WebSocket è identificata univocamente e possiede un proprio stato; ogni messaggio in ingresso è elaborato su un thread worker (annotazione @Blocking) poiché gli handler eseguono operazioni JDBC bloccanti.

Registro delle connessioni e stato

La classe ConnectionRegistry mantiene, tramite due ConcurrentHashMap, le connessioni aperte e i rispettivi ConnectionState. Lo stato di autenticazione (username e ruolo) usa campi volatile: è impostato dal LOGIN e azzerato dal LOGOUT o alla chiusura della connessione.

Transazioni ed eventi in tempo reale

Gli handler che modificano dati sono annotati @Transactional. Al termine dell'elaborazione, l'endpoint invia l'eventuale evento di broadcast dopo il commit della transazione. La classe Broadcaster invia l'evento a tutte le connessioni aperte tranne quella che lo ha originato, usando OpenConnections gestito dal framework.

Gestione degli errori

In caso di messaggio malformato, azione mancante o eccezione durante l'elaborazione, il Dispatcher restituisce una RESPONSE d'errore (VALIDATION o SERVICE_ERROR) senza compromettere le altre connessioni. La disconnessione di un client provoca la rimozione dal registro e la pulizia del relativo stato.

Strutture Dati

Le entità principali (utente, ristorante, recensione, preferito) sono rappresentate lato server da classi JPA (*UserEntity*, *RestaurantEntity*, *ReviewEntity*, *FavoriteEntity*) che riflettono direttamente le tabelle del database, e lato client/condiviso da DTO (*UserDto*, *RestaurantDto*, *ReviewDto*). La risposta del ristorante non è un'entità separata: è la colonna answer della recensione.

Scelte Algoritmiche

CorrelationId

Il server e il client comunicano attraverso WebSocket, che è full-duplex e senza accoppiamento richiesta-risposta. Le richieste e le risposte viaggiano sullo stesso socket in entrambe le direzioni e senza garanzia di ordinamento; quindi, il problema che nasce è che il client non è in grado di stabilire a quale delle proprie richieste appartenga un messaggio in arrivo. La soluzione che abbiamo deciso di adottare prevede che nell'Envelope ci sia un correlationId (UUID) che viene impostato nelle request e fa echo nelle response, cosicché ogni comunicazione abbia un Id a cui è associata, rendendo possibile al client stabilire a quale request appartenga la response in arrivo dal server.

Caching del catalogo

Il dataset di riferimento contiene circa 17.700 ristoranti, corrispondenti a poco meno di 18 MB di dati serializzati in JSON. Il trasferimento integrale del catalogo a ogni client all'avvio della sessione è stato scartato per ragioni di tempo di attesa percepito e di robustezza della connessione, in particolare nello scenario di dimostrazione in cui il backend è esposto attraverso un tunnel.

Il protocollo impone quindi che l'azione di elencazione dei ristoranti specifici obbligatoriamente una località, e il client mantiene in memoria i risultati di una sola città alla volta.

Pattern

Singleton – BackendClient

La classe BackendClient è un singleton perché garantisce che esista una sola istanza di BackendClient in tutta l'applicazione e fornisce un punto di accesso globale a tale istanza. Presenta inoltre i tre elementi fondamentali del Singleton:

- Costruttore Privato

```
private BackendClient() {  
}
```

- Istanza statica unica

```
private static final BackendClient INSTANCE = new BackendClient();
```

- Metodo pubblico di accesso

```
public static BackendClient get() {  
    return INSTANCE;  
}
```

Limiti della Soluzione Sviluppata

Il modello architetturale presenta alcune limitazioni note. In particolare:

- il backend gestisce ogni messaggio su un thread worker (operazioni JDBC bloccanti, annotazione `@Blocking`): la scalabilità è adeguata a un numero contenuto di client concorrenti, non a scenari ad alto carico;
- il broadcast degli eventi ai client è sincrono (invio sequenziale a tutte le connessioni aperte);
- non sono previsti bilanciamento del carico né esecuzione in cluster;
- lo stato di sessione è legato alla singola connessione WebSocket.

Sitografia

- <https://stackoverflow.com/questions/4165421/opencsv-csv-to-javabean>
- <https://mvnrepository.com/artifact/org.slf4j/slf4j-api>
- <https://mvnrepository.com/artifact/org.projectlombok/lombok>
- <https://mvnrepository.com/artifact/org.openjfx/javafx-controls>
- <https://mvnrepository.com/artifact/com.fasterxml.jackson.core/jackson-databind>
- <https://mvnrepository.com/artifact/org.springframework.security/spring-security-core>
- <https://quarkus.io/guides/websockets-next-tutorial>
- <https://quarkus.io/guides/security-getting-started-tutorial>
- <https://ngrok.com/docs/start>
- <https://docs.docker.com/reference/api/engine/>
- <https://www.postgresql.org/docs/>